TASK 1: News Topic Classifier (BERT)

```
# Install dependencies
!pip install transformers datasets scikit-learn -q

import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from sklearn.metrics import accuracy_score, f1_score
import numpy as np

# Load small subset of AG News (to save RAM)
dataset = load_dataset("ag_news")
dataset = dataset["train"].shuffle(seed=42).select(range(2000))  # SMALL subset

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Tokenization
def tokenize(example):
    return tokenizer(example["text"], truncation=True, padding="max_length", max_length=128)

dataset = dataset.map(tokenize, batched=True)

# Train-test split
dataset = dataset.train_test_split(test_size=0.2)

# Load model
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=4)

# Metrics
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    return {
        "accuracy": accuracy_score(labels, preds),
        "f1": f1_score(labels, preds, average="weighted")
    }

# Training config (lightweight)
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=1,  # keep small
    logging_steps=50,
    save_strategy="no"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    compute_metrics=compute_metrics
)


trainer.train()
trainer.evaluate()
```

```
Loading weights: 100%                                                    199/199 [00:00<00:00, 4654.49it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.bias                         | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.predictions.transform.dense.bias         | UNEXPECTED |
classifier.bias                              | MISSING    |
classifier.weight                            | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream ta
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as tru
  super().__init__(loader)
```

[200/200 26:11, Epoch 1/1]

Step	Training Loss
50	0.840035
100	0.408460
150	0.450296
200	0.334510

[50/50 01:53]

Training Loss	Validation Loss	Step	Accuracy	F1
0.334510	0.376269	200	0.892500	0.892931

```
{'eval_loss': 0.3762693703174591,
 'eval_accuracy': 0.8925,
 'eval_f1': 0.8929309123780006}
```

TASK 2: ML Pipeline (Churn Prediction)

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

# Load Telco Churn (Using a public URL for convenience)
url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
df = pd.read_csv(url)

# Basic Clean
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce').fillna(0)
X = df.drop(['customerID', 'Churn'], axis=1)
y = df['Churn'].apply(lambda x: 1 if x == 'Yes' else 0)

# Identify features
numeric_features = X.select_dtypes(include=['int64', 'float64']).columns
categorical_features = X.select_dtypes(include=['object']).columns

# 2. Construct Pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier())
])

# 3. Hyperparameter Tuning
param_grid = {
    'classifier__n_estimators': [50, 100],
    'classifier__max_depth': [None, 10]
}

grid_search = GridSearchCV(full_pipeline, param_grid, cv=3, scoring='accuracy')
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

grid_search.fit(X_train, y_train)

# 4. Export
joblib.dump(grid_search.best_estimator_, 'churn_pipeline_model.joblib')
print(f"Best Score: {grid_search.best_score_}")
```

```
Best Score: 0.7955271565495208
```

Task 3: Multimodal ML – Housing Price Prediction Using Images + Tabular Data

```
!pip install torch torchvision pandas -q

import torch
import torch.nn as nn
import torchvision.models as models
from torchvision import transforms
from PIL import Image
import pandas as pd
import numpy as np

# Dummy tabular data (replace with real dataset)
tabular_data = pd.DataFrame({
    "beds": [2,3,4],
    "bath": [1,2,3],
    "price": [100,200,300]
})

# Simple CNN feature extractor (lightweight)
cnn = models.resnet18(pretrained=True)
cnn.fc = nn.Identity()  # remove classifier

# Image transform
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor()
])

# Example image feature extraction
def extract_features(img_path):
    img = Image.open(img_path).convert("RGB")
    img = transform(img).unsqueeze(0)
    with torch.no_grad():
        features = cnn(img)
    return features.numpy().flatten()

# Example combine
image_features = np.random.rand(3, 512)  # placeholder for demo
tabular_features = tabular_data[["beds","bath"]].values

# Combine both
X = np.hstack([tabular_features, image_features])
y = tabular_data["price"].values

# Simple regression model
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(n_estimators=50)
model.fit(X, y)

print("Model trained on multimodal data")
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f370
100%|██████████| 44.7M/44.7M [00:00<00:00, 183MB/s]
Model trained on multimodal data
```

Task 4: Context-Aware Chatbot Using LangChain or RAG

```
!pip install faiss-cpu sentence-transformers transformers -q
```

```
import faiss
```

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from transformers import pipeline

# ----------------------------
# 1. Create Knowledge Base
# ----------------------------
docs = [
    "Machine learning is a field of artificial intelligence.",
    "Python is widely used for data science and AI.",
    "Transformers are deep learning models used in NLP tasks.",
    "LangChain is a framework for building LLM applications."
]

# ----------------------------
# 2. Load Embedding Model (lightweight)
# ----------------------------
embedder = SentenceTransformer('all-MiniLM-L6-v2')

# Convert docs → vectors
doc_vectors = embedder.encode(docs)

# ----------------------------
# 3. Create FAISS Index
# ----------------------------
dimension = doc_vectors.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(np.array(doc_vectors))

# ----------------------------
# 4. Load Lightweight LLM
# ----------------------------
generator = pipeline(
    "text-generation",
    model="google/flan-t5-small",
    max_new_tokens=100
)

# ----------------------------
# 5. Retrieval Function
# ----------------------------
def retrieve(query, k=2):
    query_vec = embedder.encode([query])
    distances, indices = index.search(np.array(query_vec), k)
    return [docs[i] for i in indices[0]]

# ----------------------------
# 6. RAG Response Function
# ----------------------------
def generate_answer(query):
    context = retrieve(query)

    prompt = f"""
    Answer the question using the context below.

    Context:
    {context}

    Question:
    {query}
    """

    result = generator(prompt)
    return result[0]['generated_text']

# ----------------------------
# 7. Chat Loop (Context-aware simulation)
# ----------------------------
chat_history = []

print("Chatbot ready! Type 'exit' to stop.\n")

while True:
    query = input("You: ")

    if query.lower() == "exit":
        break

    answer = generate_answer(query)

    chat_history.append((query, answer))
```

```
    print("Bot:", answer)
```

```
Loading weights: 100%|| 103/103 [00:00<00:00, 2836.52it/s]
model.safetensors: 100%|| 308M/308M [00:03<00:00, 158MB/s]
Loading weights: 100%|| 190/190 [00:00<00:00, 2189.44it/s]
[transformers] The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both
generation_config.json: 100%|| 147/147 [00:00<00:00, 9.71kB/s]
tokenizer_config.json: 2.54k/? [00:00<00:00, 85.4kB/s]
spiece.model: 100%|| 792k/792k [00:00<00:00, 2.63MB/s]
tokenizer.json: 2.42M/? [00:00<00:00, 34.0MB/s]
special_tokens_map.json: 2.20k/? [00:00<00:00, 209kB/s]
[transformers] Passing `generation_config` together with generation-related arguments=({'max_new_tokens'}) is deprecated and
[transformers] The model 'T5ForConditionalGeneration' is not supported for text-generation. Supported models are ['PeftModel
Chatbot ready! Type 'exit' to stop.

You: Hi
[transformers] Both `max_new_tokens` (=100) and `max_length`(=20) seem to have been set. `max_new_tokens` will take preceder
Bot:
    Answer the question using the context below.

    Context:
    ['Machine learning is a field of artificial intelligence.', 'Python is widely used for data science and AI.']

    Question:
    Hi

You: exit
```

Task 5: Auto Tagging Support Tickets (LLM)

```
!pip install langchain langchain-openai -q
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 98.6/98.6 kB 2.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 542.4/542.4 kB 12.9 MB/s eta 0:00:00
```

```
!pip install transformers -q

from transformers import pipeline

# Load zero-shot classifier (lightweight)
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")

# Sample ticket
ticket = "My internet is not working and keeps disconnecting"

# Candidate labels
labels = ["network issue", "billing", "technical support", "account issue"]

# Predict
result = classifier(ticket, labels)

# Top 3 tags
top3 = list(zip(result["labels"][:3], result["scores"][:3]))

print("Top 3 Tags:")
for tag, score in top3:
    print(tag, round(score, 3))
```

```
config.json: 1.15k/? [00:00<00:00, 28.7kB/s]
model.safetensors: 100%|| 1.63G/1.63G [00:16<00:00, 153MB/s]
Loading weights: 100%|| 515/515 [00:00<00:00, 4659.83it/s]
tokenizer_config.json: 100%|| 26.0/26.0 [00:00<00:00, 905B/s]
vocab.json: 899k/? [00:00<00:00, 8.44MB/s]
merges.txt: 456k/? [00:00<00:00, 8.14MB/s]
tokenizer.json: 1.36M/? [00:00<00:00, 12.5MB/s]
Top 3 Tags:
network issue 0.868
technical support 0.072
account issue 0.051
```